



INTRODUCTION TO SOFTWARE ENGINEERING

Part II – Software Process Requirement Analysis & Specification

Software Engineering Department
College of Information and Communication Technology

Content

- Requirements analysis & specification (Definition)
- Design
- Implementation (Programming)
- Testing
- Delivery and Maintenance

Content

- The Requirements Process
- Requirements Elicitation
- Types of Requirements
- Characteristics of Requirements
- Modeling Notations
- Requirements and Specification Languages
- Requirements Documentation
- Validation and Verification

Requirements Process

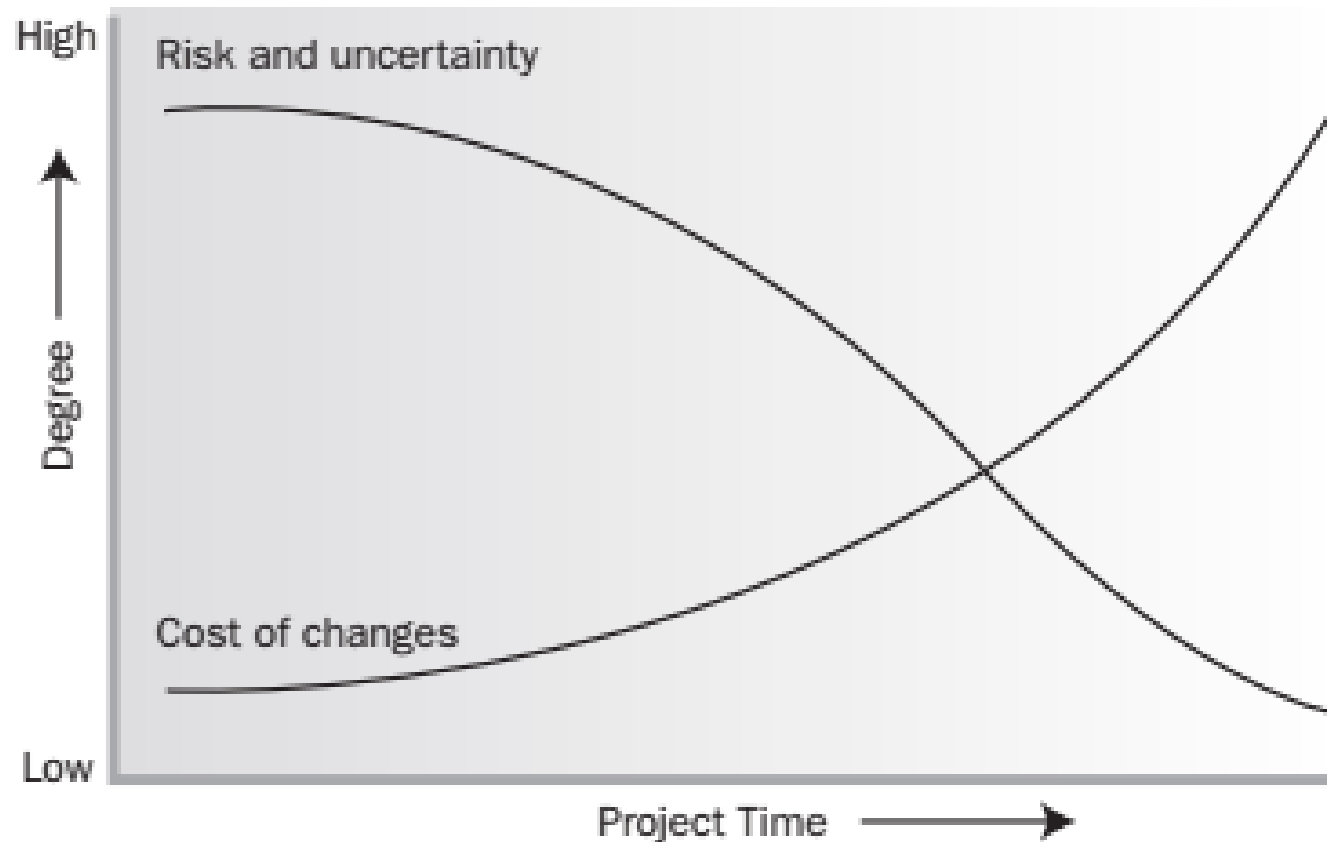
- A *requirement* is an expression of desired behavior
- A requirement deals with
 - objects or entities
 - the state they can be in
 - functions that are performed to change states or object characteristics
- Requirements focus on the customer needs, not on the solution or implementation
 - designate *what* behavior, without saying *how* that behavior will be realized

Requirements Process

Why are requirements important?

- Top factors that caused project to fail
 - Incomplete requirements
 - Lack of user involvement
 - Unrealistic expectations
 - Lack of executive support
 - Changing requirements and specifications
 - Lack of planning
 - System no longer needed
- Some part of the requirements process is involved in almost all of these causes
- Requirements error can be expensive if not detected early

Requirements Process

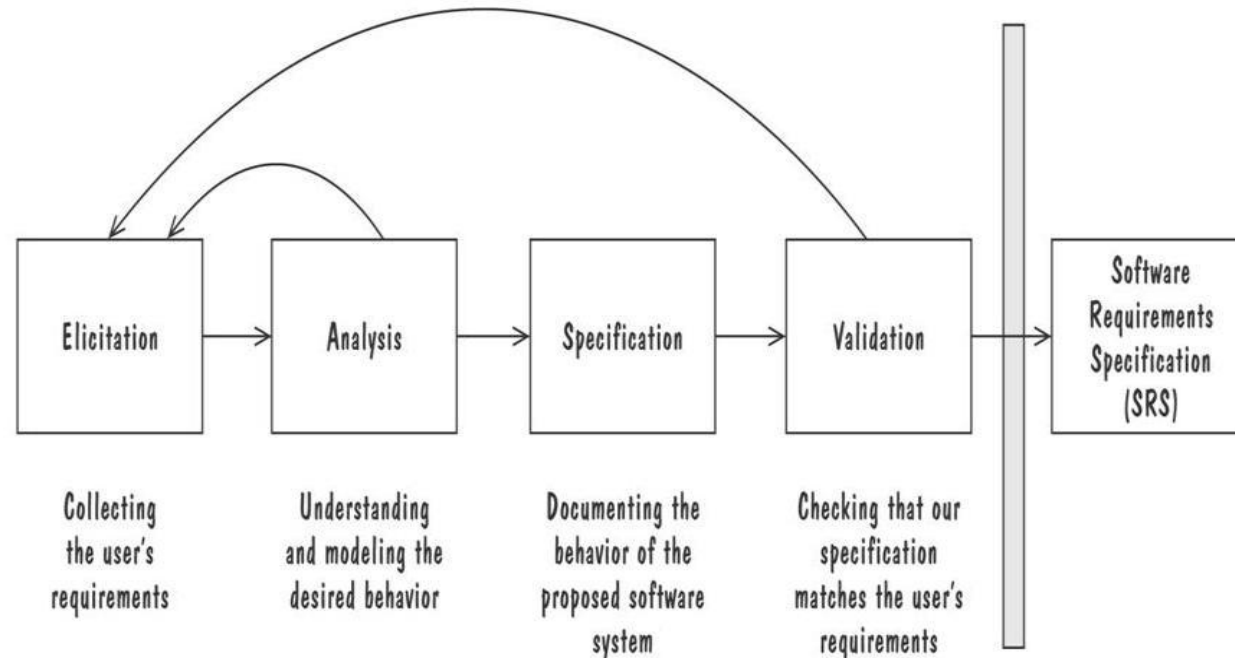


Impact of Variable Based on Project time

Requirements Process

Process for capturing requirements

- Performed by the req. analyst or system analyst
- The final outcome is a Software Requirements Specification (**SRS**) document



Content

- The Requirements Process
- Requirements Elicitation
- Types of Requirements
- Characteristics of Requirements
- Modeling Notations
- Requirements and Specification Languages
- Requirements Documentation
- Validation and Verification

Requirements Elicitation

- Customers do not always understand what their needs and problems are
- It is important to discuss the requirements with everyone who has a stake in the system
- Come up with agreement on what the requirements are
 - If we cannot agree on what the requirements are, then the project is doomed to fail

Requirements Elicitation

Stakeholders

- Clients: pay for the software to be developed
- Customers: buy the software after it is developed
- Users: use the system
- Domain experts: familiar with the problem that the software must automate
- Market Researchers: conduct surveys to determine future trends and potential customers
- Lawyers or auditors: familiar with government, safety, or legal requirements
- Software engineers or other technology experts

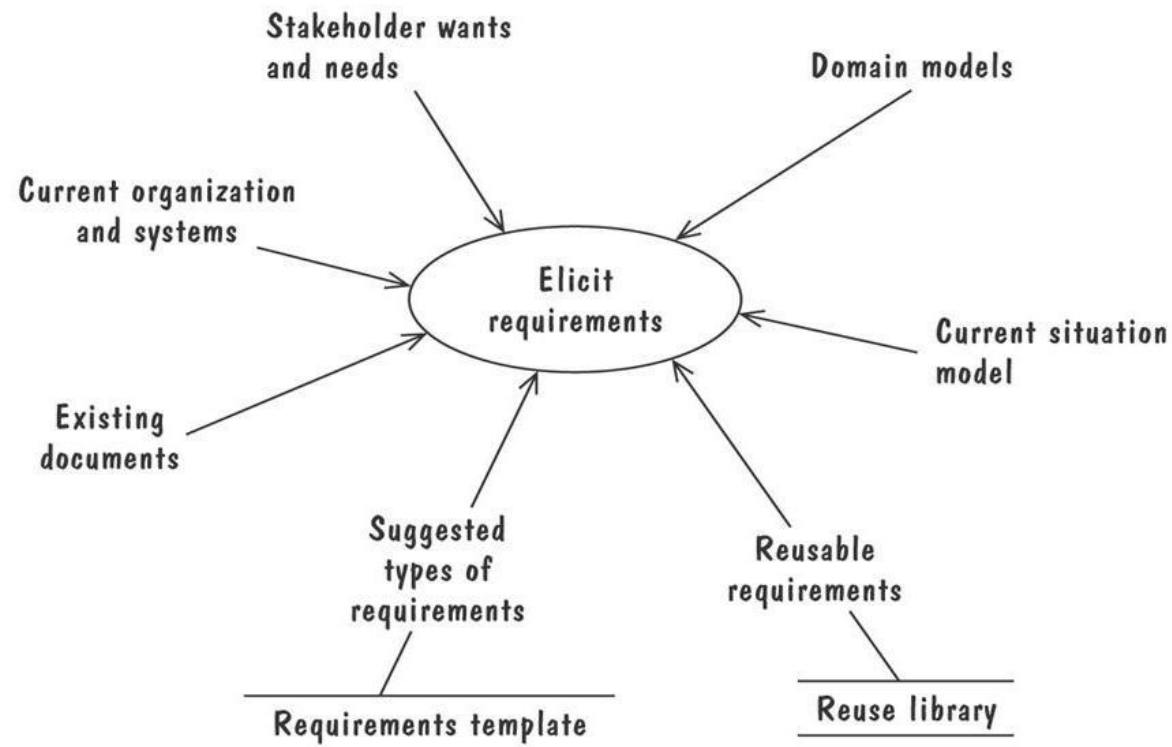
Requirements Elicitation

Means of Eliciting Requirements

- Interviewing stake holders
- Reviewing available documentations
- Observing the current system (if one exists)
- Apprenticing with users to learn about user's task in more details
- Interviewing user or stakeholders in groups
- Using domain specific strategies, such as Joint Application Design, or PIECES
- Brainstorming with current and potential users

Requirements Elicitation

- The Volere requirements process model suggests some additional sources for requirements



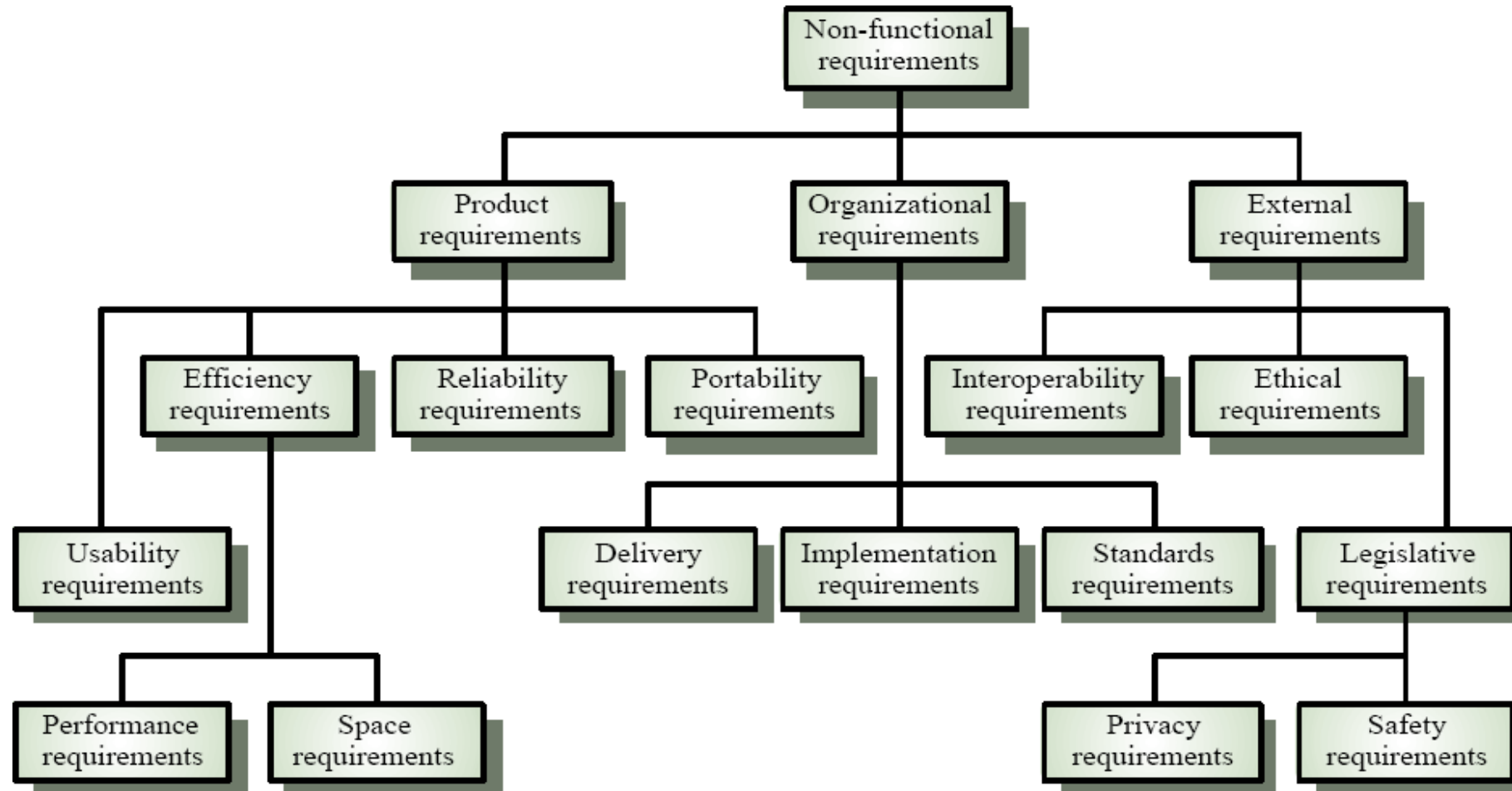
*Pfleeger and Atlee, Software Engineering:
Theory and Practice*

Types of Requirements

- **Functional requirement**: describes required behaviour in terms of required activities
 - **Quality requirement** or **non-functional requirement**: describes some quality characteristic that the software must possess
 - **Design constraint**: a design decision such as choice of platform or interface components
 - **Process constraint**: a restriction on the techniques or resources that can be used to build the system
- 

Types of Requirements

- Non-functional Requirements



Types of Requirements

Resolving Conflicts

- Different stakeholder has different set of requirements
 - potential conflicting ideas
- Need to prioritize requirements
- Prioritization might separate requirements into three categories
 - essential: absolutely must be met
 - desirable: highly desirable but not necessary
 - optional: possible but could be eliminated

Characteristics of Requirements

- Correct
- Consistent
- Unambiguous
- Complete
- Feasible
- Relevant
- Testable
- Traceable

Content

- The Requirements Process
- Requirements Elicitation
- Types of Requirements
- Characteristics of Requirements
- Modeling Notations
- Requirements and Specification Languages
- Requirements Documentation
- Validation and Verification

Modeling Notations

- It is important to have standard notations for modeling, documenting, and communicating decisions
- Modeling helps us to understand requirements thoroughly
 - Holes in the models reveal unknown or ambiguous behavior
 - Multiple, conflicting outputs to the same input reveal inconsistencies in the requirements

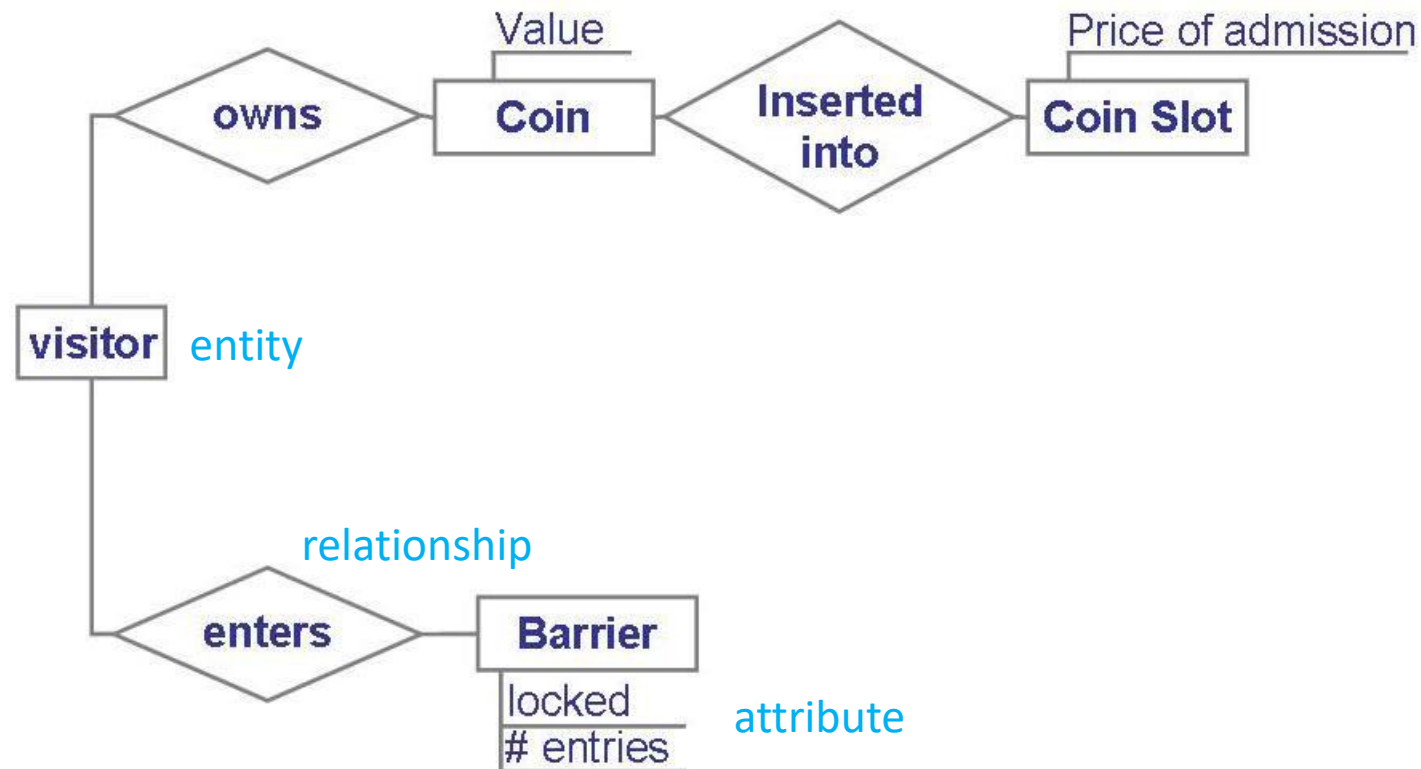
Modeling Notations

Entity-Relationship Diagrams (ERD)

- A popular graphical notational paradigm for representing **conceptual models**
- Has three core constructs
 - An *entity*: depicted as a rectangle, represents a collection of real-world objects that have common properties and behaviors
 - A *relationship*: depicted as an edge between two entities, with diamond in the middle of the edge specifying the type of relationship
 - An *attribute*: an annotation on an entity that describes data or properties associated with the entity

Modeling Notations

- Example: Entity - Relationship diagram of turnstile problem



Modeling Notations

- ER diagrams are popular because
 - they provide an **overview of the problem** to be addressed
 - the view is **relatively stable** when changes are made to the problem's requirements
- ER diagram is likely to be used to model a problem early in the requirements process

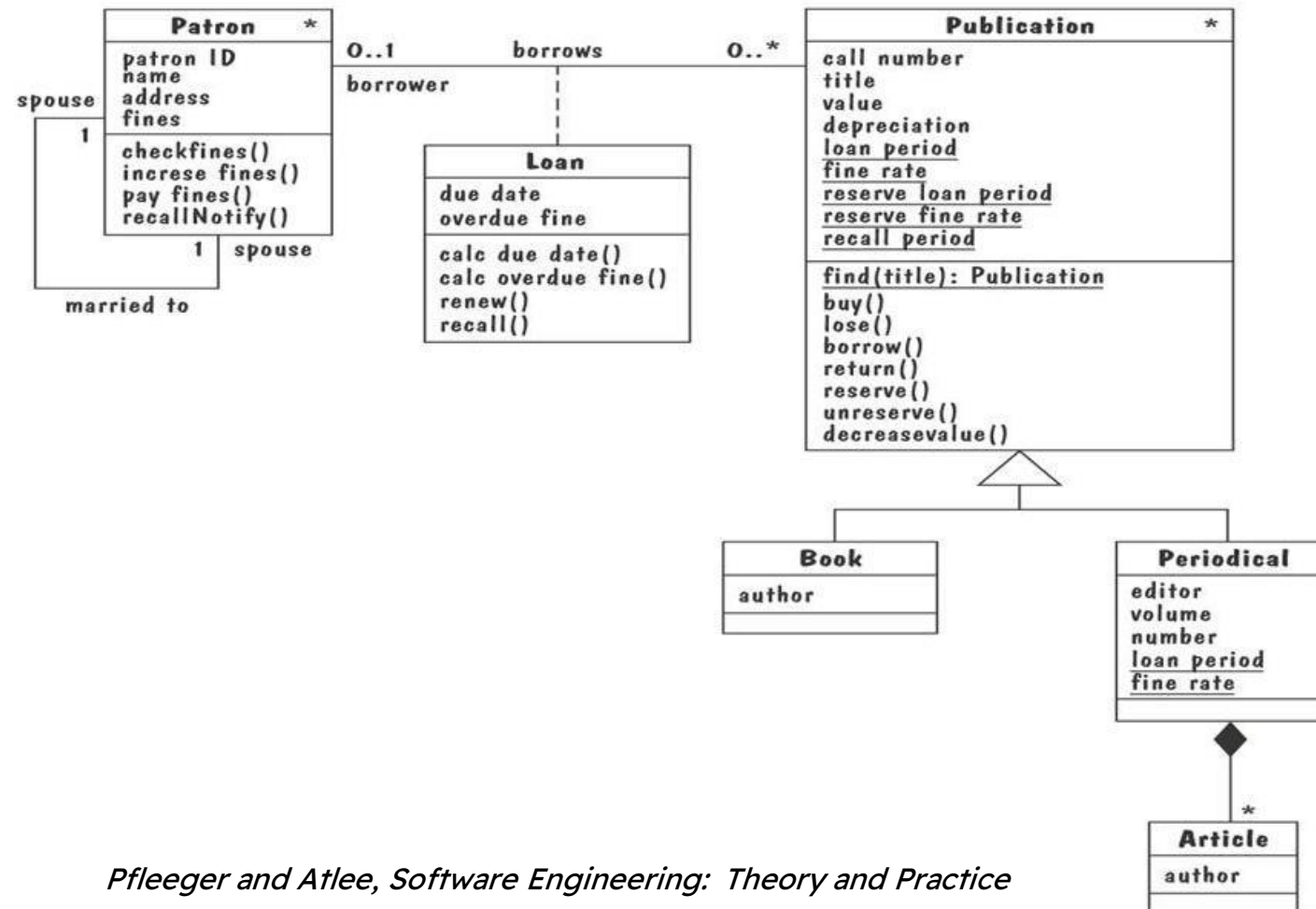
Modeling Notations

Unified Modeling Language Class Diagram is an example of ER Diagrams

- Unified Modeling Language (UML) is a collection of notations used to document software specifications and designs
- It represents a system in terms of
 - *objects*: akin to entities, organized in classes that have an inheritance hierarchy
 - *methods*: actions on the object's variables
- The **class diagram** is the flagship model in any UML specification
 - A sophisticated **ER diagram** relating the classes (entities) in the specification

Modeling Notations

- Example: UML Class Diagram of Library Problem



Modeling Notations

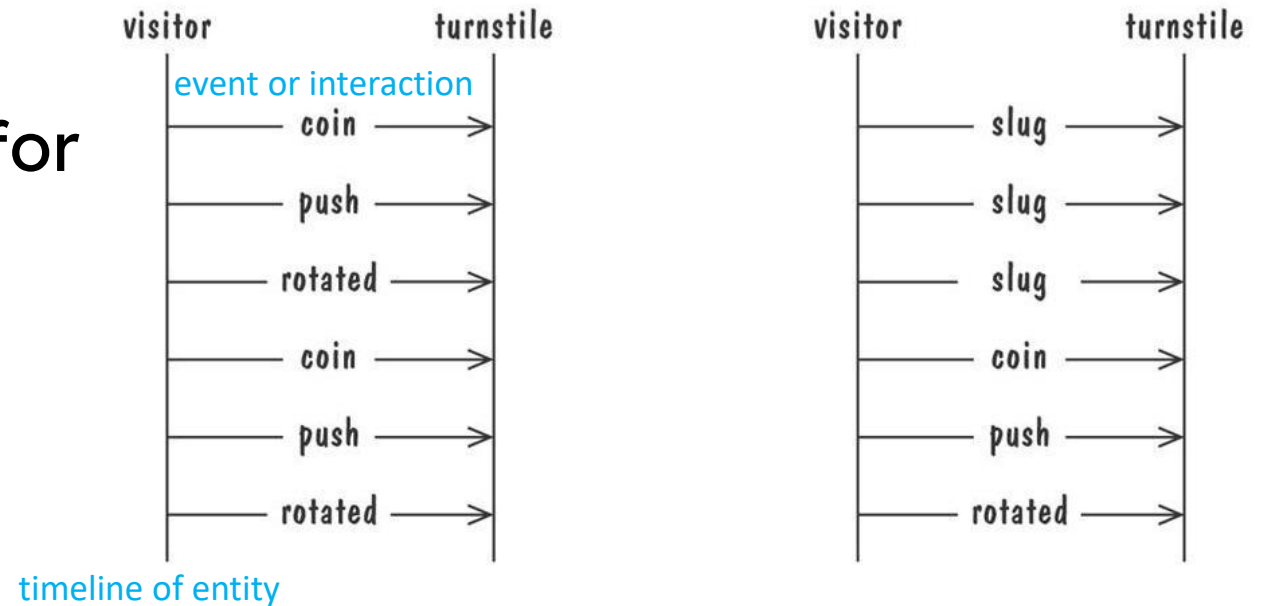
Event Traces

- A graphical description of a sequence of events that are exchanged between real-world entities
 - *Vertical line*: the timeline of distinct entity, whose name appear at the top of the line
 - *Horizontal line*: an event or interaction between the two entities bounding the line
 - Time progresses from top to bottom
- Each graph depicts a single trace, representing one of several possible behaviors
- Traces have a semantic that is *relatively precise, simple and easy to understand*

Modeling Notations

Event Traces

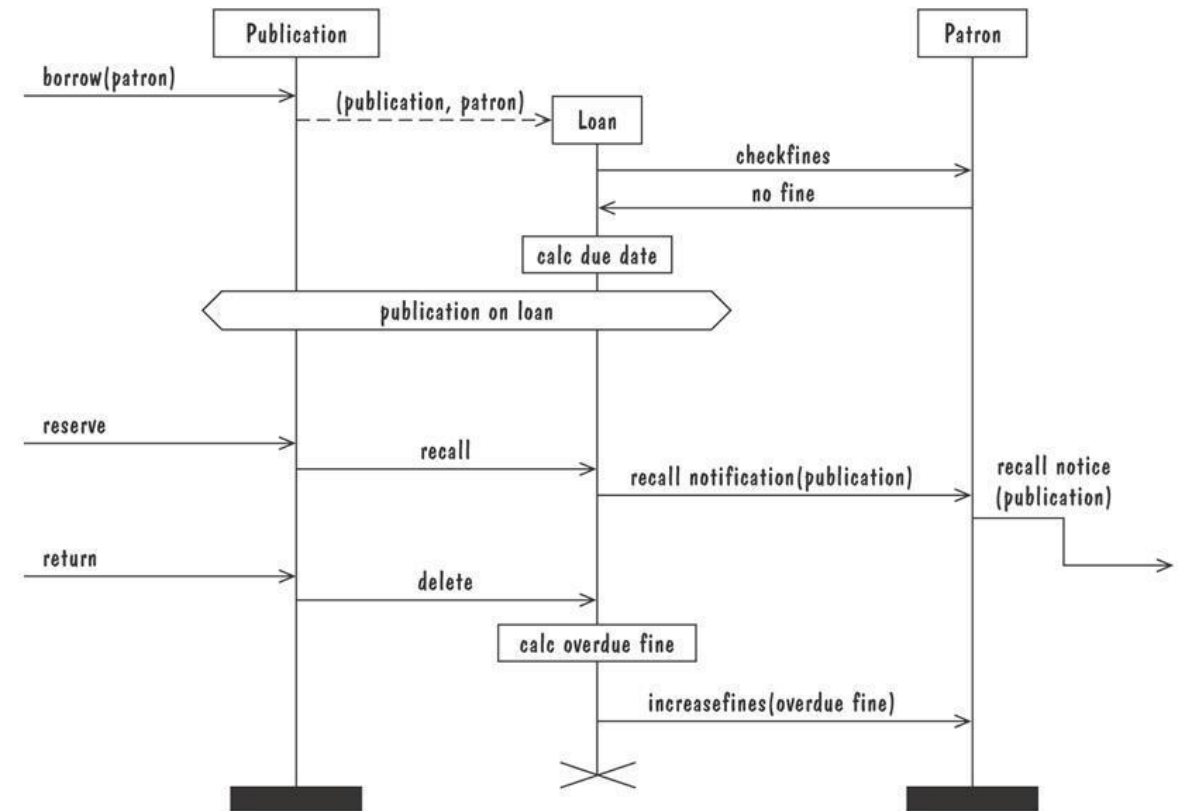
- Example: Graphical representation of two traces for the turnstile problem
 - trace on the left represents typical behavior
 - trace on the right shows exceptional behavior



Pfleegeer and Atlee, Software Engineering: Theory and Practice

Modeling Notations

- **Message Sequence Chart** is an example event traces
 - Example: Message sequence chart for library loan transaction



Modeling Notations

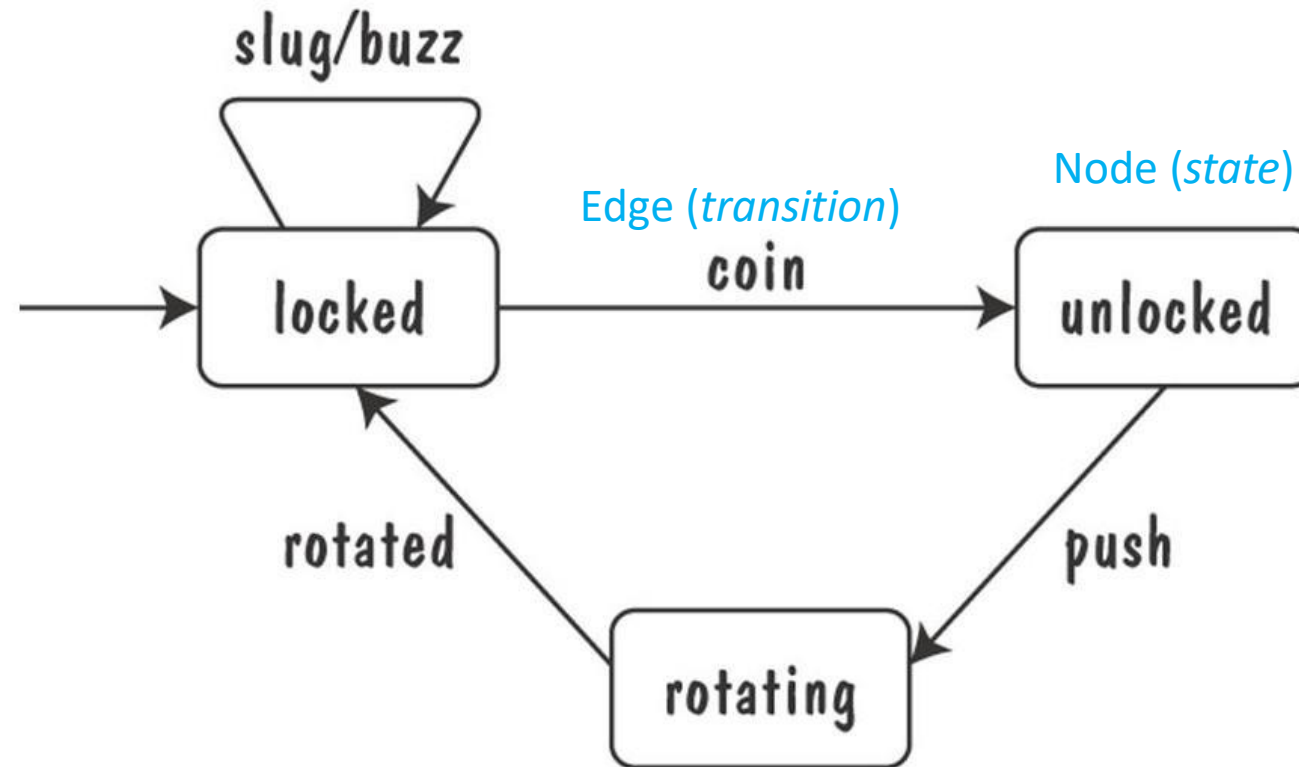
State Machines

- A graphical description of all dialog between the system and its environment
 - Node (*state*) represents a stable set of conditions that exists between event occurrences
 - Edge (*transition*) represents a change in behavior or condition due to the occurrence of an event
- Useful both for specifying dynamic behavior and for describing how behavior should change in response to the history of events that have already occurred

Modeling Notations

State Machines

- Example: Finite state machine model of the turnstile problem



Modeling Notations

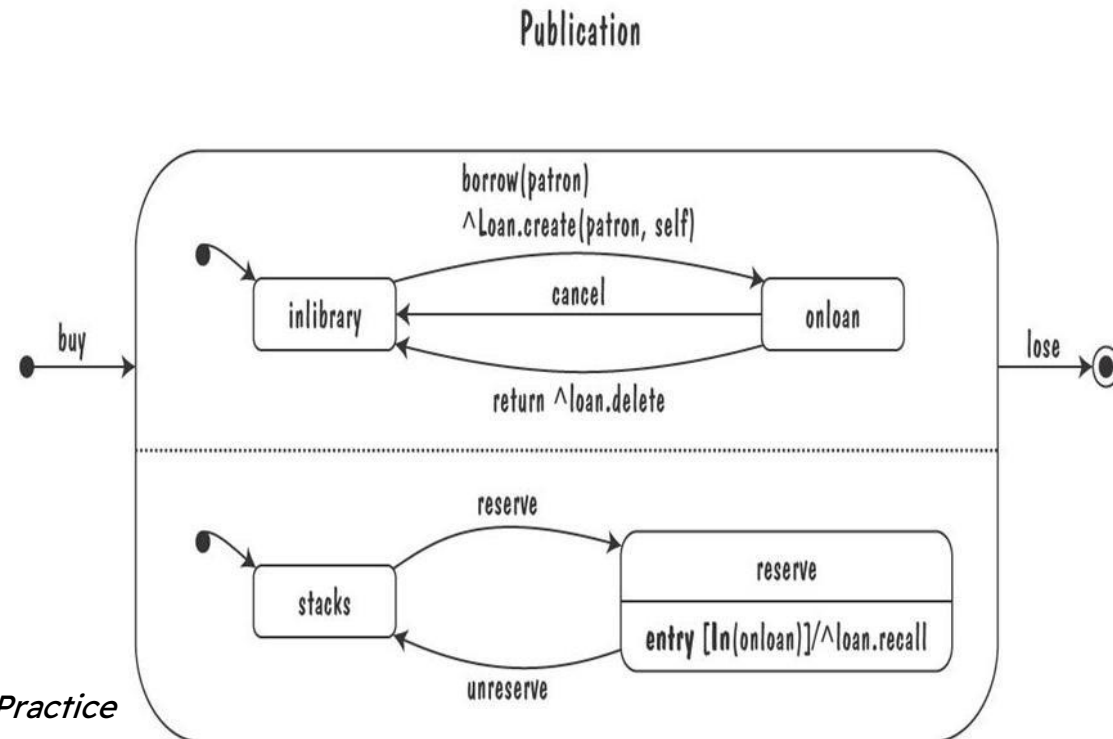
State Machines

- A path: starting from the machine's initial state and following transitions from state to state
 - A trace of observable events in the environment
- Deterministic state machine: for every state and event there is a unique response

Modeling Notations

UML Statechart Diagrams is an example of state machines

- Example: The UML statechart diagram for the `Publication` class from the Library class model



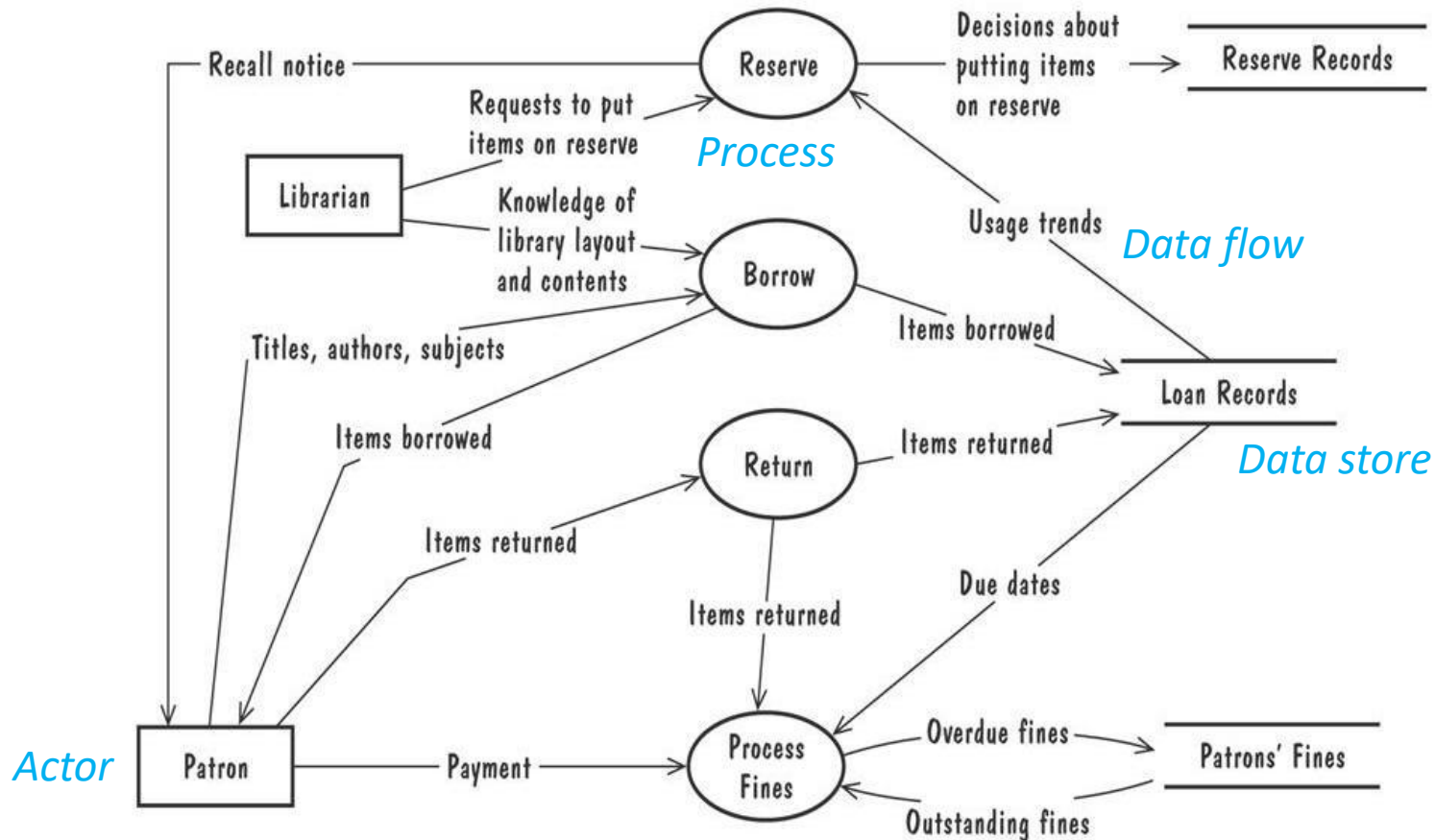
Modeling Notations

Data-Flow Diagrams

- ER diagram, event trace, state machines depict only lower-level behaviors
- A **data-flow diagram (DFD)** models functionality and the flow of data from one function to another
 - A bubble represents a *process*
 - An arrow represents *data flow*
 - A *data store*: a formal repository or database of information
 - Rectangles represent *actors*: entities that provide input data or receive the output result

Modeling Notations

- Example: A high-level data-flow diagram for the library problem



*Pfleeger and Atlee, Software Engineering:
Theory and Practice*

Modeling Notations

Data-Flow Diagrams

- Advantage:

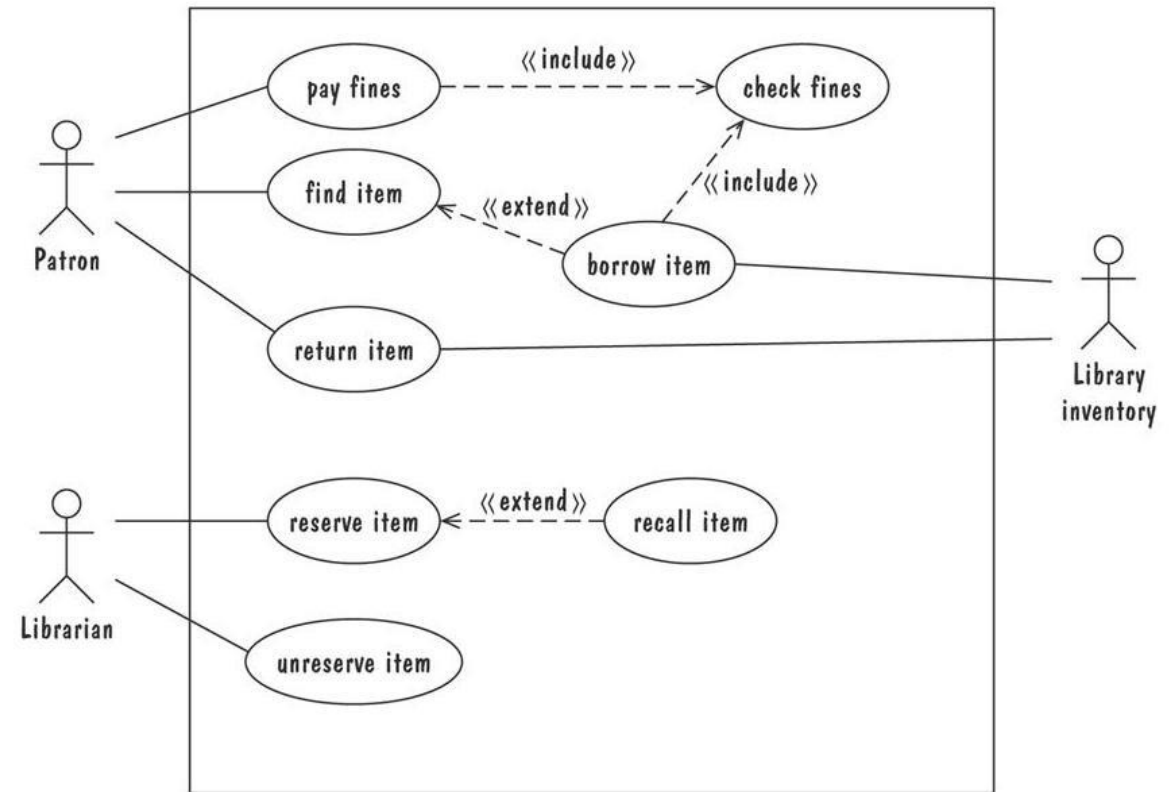
- Provides an intuitive model of a proposed **system's high-level functionality** and of the data dependencies among various processes

- Disadvantage:

- Can be aggravatingly ambiguous to a software developer who is less familiar with the problem being modeled

Modeling Notations

- **Use Cases** is an example of data-flow diagrams
 - Example: Library use cases including borrowing a book, returning a borrowed book, and paying a library fine



Pfleeger and Atlee, Software Engineering: Theory and Practice

Modeling Notations

- Functions and Relations
 - Functions and Relations Example: Decision Tables
 - Functions and Relations Example: Parnas Tables
- Logic
 - Logic Example: Object Constrain Language (OCL)
 - Logic Example: Z
- Algebraic Specifications
 - Algebraic Specifications Example: SDL Data

Content

- The Requirements Process
- Requirements Elicitation
- Types of Requirements
- Characteristics of Requirements
- Modeling Notations
- Requirements and Specification Languages
- Requirements Documentation
- Validation and Verification

Requirements and Specification Languages

Unified Modeling Language (UML)

- Combines multiple notation paradigms
- Eight graphical modeling notations, and the OCL constrain language, **including**
 - Use-case diagram (a high-level DFD)
 - Class diagram (an ER diagram)
 - Sequence diagram (an event trace)
 - Collaboration diagram (an event trace)
 - Statechart diagram (a state-machine model)
 - OCL properties (logic)

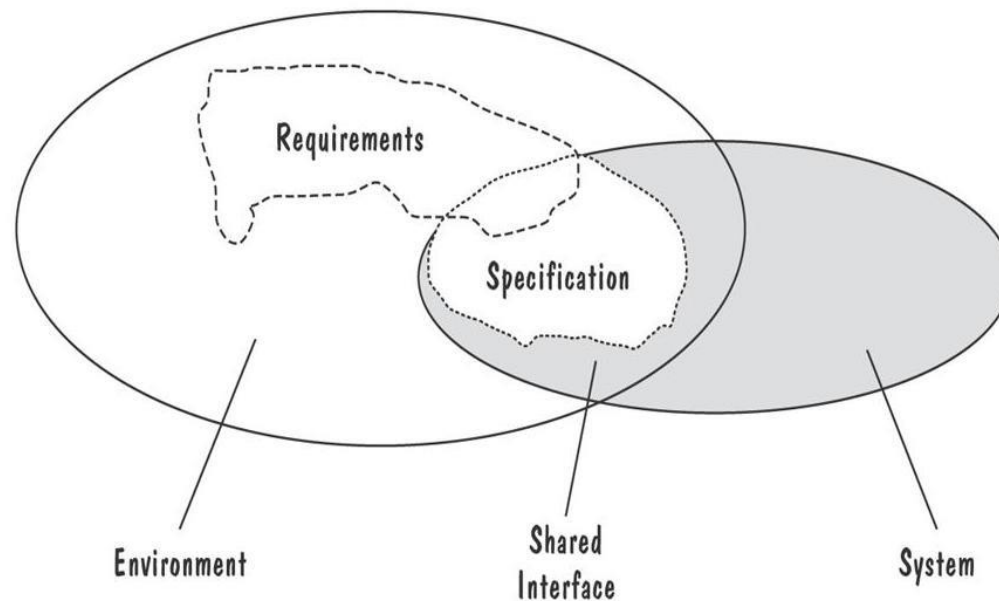
Requirements and Specification Languages

Specification and Description Language (SDL)

- Standardized by the International Telecommunication Union
- Specifies the behavior of real-time, concurrent, distributed processes that communicate with each other via unbounded message queues
- **Comprises**
 - SDL system diagram (a DFD)
 - SDL block diagram (a DFD)
 - SDL process diagram (a state-machine model)
 - SDL data type (algebraic specification)
- Often accompanied by a set of Message Sequence Chart (MSC)

Requirements Documentation

- **Requirements definition:** a complete listing of everything the customer wants to achieve
 - Describing the entities in the environment where the system will be installed
- **Requirements specification:** restates the requirements as a specification of how the proposed system shall behave



- Requirements defined anywhere within the environment's domain, including the system's interface
- Specification restricted only to the intersection between environment and system domain

- Requirements definition

- 1. The a mental health care patient management system (MHC-PMS) shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

- Requirements specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost, and the prescribing clinics shall be generated.
 - 1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
 - 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed, and the total cost of the prescribed drugs.
 - 1.4 If drugs are available in different dose units (e.g., 10 mg, 20 mg) separate reports shall be created for each dose unit.
 - 1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.

Requirements Documentation

Requirement Definition: steps documenting process

- Outline the general purpose and scope of the system, including relevant benefits, objectives, and goals
- Describe the background and the rationale behind proposal for new system
- Describe the essential characteristics of an acceptable solution
- Describe the environment in which the system will operate
- Outline a description of the proposal, if the customer has a proposal for solving the problem
- List any assumptions we make about how the environment behaves

Requirements Documentation

Requirements Specification: steps documenting process

- Describe all inputs and outputs in detail, including
 - the sources of inputs
 - the destinations of outputs,
 - the value ranges
 - data format of inputs and output data
 - data protocols
 - window formats and organizations
 - timing constraint
- Restate the required functionality in terms of the interfaces' inputs and outputs
- Devise fit criteria for each of the customer's quality requirements

Requirements Documentation

IEEE Standard for SRS Organized by Objects

1. Introduction to the Document
 - 1.1 Purpose of the Product
 - 1.2 Scope of the Product
 - 1.3 Acronyms, Abbreviations, Definitions
 - 1.4 References
 - 1.5 Outline of the rest of the SRS
2. General Description of Product
 - 2.1 Context of Product
 - 2.2 Product Functions
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and Dependencies

3. Specific Requirements
 - 3.1 External Interface Requirements
 - 3.1.1 User Interfaces
 - 3.1.2 Hardware Interfaces
 - 3.1.3 Software Interfaces
 - 3.1.4 Communications Interfaces
 - 3.2 Functional Requirements
 - 3.2.1 Class 1
 - 3.2.2 Class 2
 - ...
 - 3.3 Performance Requirements
 - 3.4 Design Constraints
 - 3.5 Quality Requirements
 - 3.6 Other Requirements
4. Appendices

Validation and Verification

- In requirements **validation**, we check that our requirements definition accurately reflects the customer's needs
- In **verification**, we check that one document or artifact conforms to another
- **Verification** ensures that we build the **system right**, whereas **validation** ensures that we build the **right system**

List of techniques to validate and verify requirements

Validation	Walkthroughs Readings Interviews Reviews Checklists Models to check functions and relationships Scenarios Prototypes Simulation Formal inspections
Verification	Cross-referencing Simulation Consistency checks Completeness checks Check for unreachable states of transitions
Checking	Model checking Mathematical proofs

Validation and Verification

Requirements Review

- Review the stated goals and objectives of the system
- Compare the requirements with the goals and objectives
- Review the environment in which the system is to operate
- Review the information flow and proposed functions
- Assess and document the risk, discuss and compare alternatives
- Testing the system: how the requirements will be revalidated as the requirements grow and change

Validation and Verification

Verification

- Check that the requirements-specification document corresponds to the requirements-definition
- Make sure that if we implement a system that meets the specification, then the system will satisfy the customer's requirements
- Ensure that each requirement in the definition document is traceable to the specification

Q&A

References

- Shari Lawrence Pfleeger, Joanne M. Atlee, Software Engineering theory and practice, Pearson Prentice Hall, 4th edition, 2010.
- Hiep Xuan Huynh, Lan Phuong Phan, Introduction to Software Engineering, Can Tho university publishing house, 2010.